# GENRESEARCH-SERVIDOR

Memória, armazenamento, sincronização e desempenho de aplicações de IA generativa

Mariana Silva Costa

Sistemas Operacionais - 2026.2

# PROBLEMA E CENÁRIO

Aplicações atuais de IA generativa e Recuperação Aumentada por Geração (RAG) impõem altas demandas ao Sistema Operacional. Estas aplicações consomem muita memória física para contextos, utilizam memória virtual para arquivos grandes e realizam operações de I/O pesadas na leitura de índices vetoriais. Elas também dependem fortemente de concorrência e filas.

O projeto busca responder como a memória principal, memória virtual, armazenamento, caches, filas compartilhadas e sincronização afetam o desempenho e a estabilidade de uma aplicação RAG.

# AMBIENTE

**Tipo de ambiente:** Nativo

**Sistema Operacional:** Fedora Linux 44 (Workstation Edition)

**Kernel:** Linux 7.2.9-200.fc44.x86_64

**CPU:** AMD Ryzen 5 5600GT with Radeon Graphics (6 núcleos, 12 threads)

**RAM:** 15 GB Total (8,3 GB disponíveis para novas alocações)

**Armazenamento:** SSD NVMe 512GB (nvme0n1) com 337 GB livres

**Swap:** 8 GB em /dev/zram0 (apenas 14 MB em uso)

**CPU:** Gráficos integrados Radeon (compartilham RAM do sistema)

**Sistema de arquivos:** Btrfs (diretório raiz e de utilizador partilham o mesmo subvolume no armazenamento)

# ARQUITETURA

Baseada no fluxo de produtor-consumir.

**Ingestão e Vetorização:** o sistema lê documentos, divide-os em pedaços e cria embeddings utilizando um hash determinístico para teste de estresse do SO.
**Armazenamento:** os textos são salvos em SQLite e o índice de vetores no formato NumPy.
**Fila Compartilhada:** as perguntas recebidas entram numa fila controlada por semáforos para evitar sobrecargas.
**Recuperação e Resposta:** threads consumidoras recolhem as perguntas, calculam a similaridade vetorial, recuperam os textos do banco e geram a resposta final.

# DADOS E CONFIGURAÇÕES

O dataset utilizado, extraído do Kaggle, contém avaliações de produtos da loja Sephora.

Foram processados **5000 documentos**, resultando em **6062 trechos** divididos.

Os vetores possuem **dimensão de 1024** e ficam **armazenados no disco**.

Para evitar retrabalho de consultas, o sistema utiliza uma **Cache na memória RAM configurada com a política LRU**.
Quando a capacidade do cache é atingida, a busca mais antiga é descartada para dar espaço às novas.

# PROCESSOS E RECURSOS

A concorrência da aplicação foi configurada por meio da biblioteca **threading**.

O sistema cria threads separadas para envio das requisições (**produtores**) e para processamento (**consumidores**).

O principal recurso compartilhado é a fila (collections.deque), dimensionada com capacidade limitada para observação de **bloqueio produtivo** (backpressure) durante **saturações de I/O.**

O recurso de cache LRU também é compartilhado entre todas as threads trabalhadoras.

# EXPERIMENTOS

Toda configuração testada foi repetida 3 vezes para eliminar anomalias nos resultados.

O experimento C1 e C2 comparou a **leitura convencional** de arquivos com **chamadas de mapeamento de memória (mmap)**.

O experimento C5 testou a **escalabilidade** variando a restrição de memória e o tamanho do corpus de 315MB a 3157MB.

A métrica e dados básicos (**tempo, RSS e faltas de página**) foram capturados utilizando a ferramenta time, módulo **resource do Python, vmstat e iostat**.

# RESULTADOS E GRÁFICOS

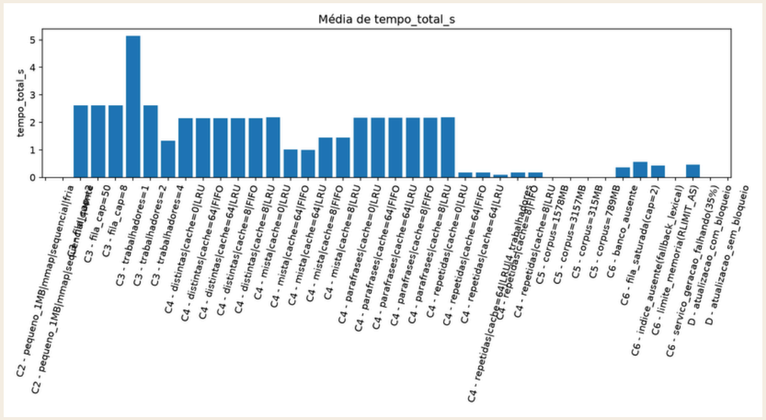
No experimento de concorrência, o aumento de 1 para 4 trabalhadores **reduziu o tempo de execução** de 5.137s para 1.329s.

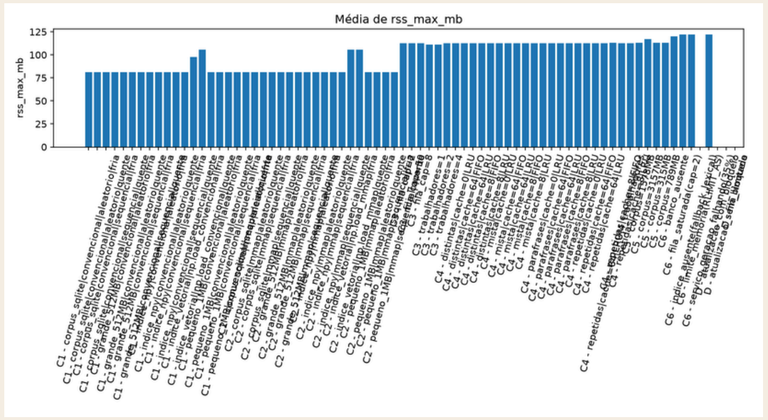
A vazão **subiu de forma linear**, partindo de 38.93 requisições por segundo para 150.5 requisições por segundo.

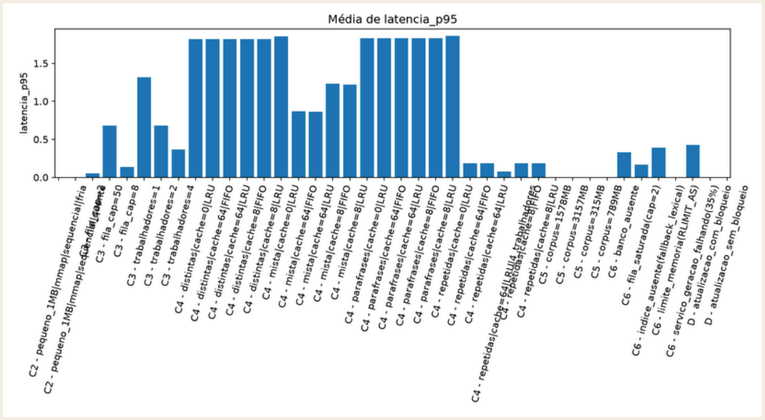
Gráficos demonstraram que **consultas repetidas** em caches de 8 ou 64 itens obtiveram 55 acertos e **pouparam** 0.9167s de execução.

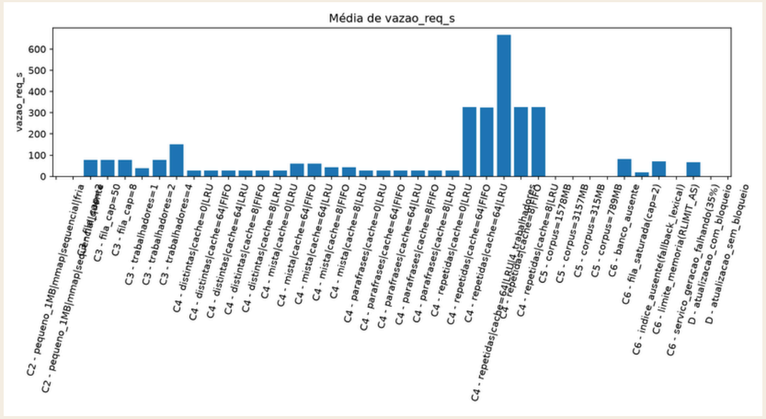
**Consultas mistas** (cache de 64 itens com LRU) tiveram 32 acertos e 28 erros.

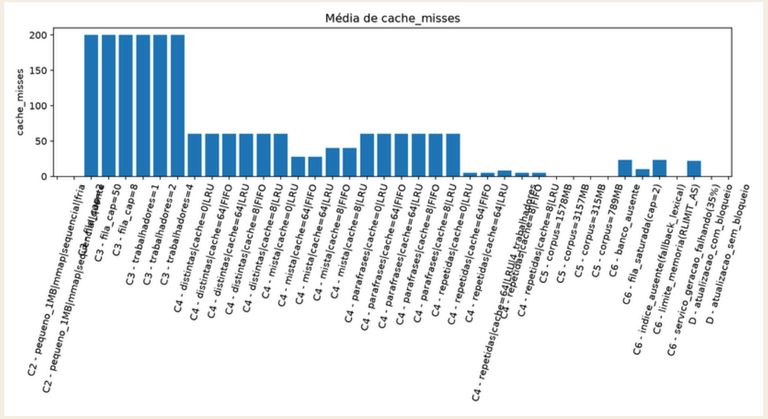
A taxa de acerto no cache foi zero para **pesquisas com paráfrases ou entradas distintas**, independentemente do uso de LRU ou FIFO.

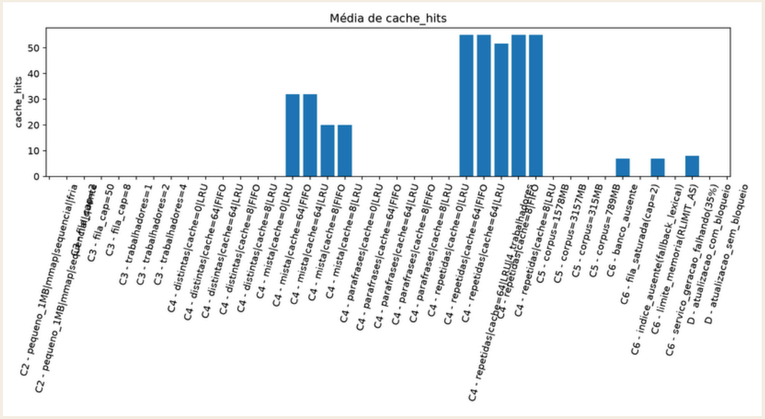

Média de tempo_total_s
tempo_total_s


Média de rss_max_mb
rss_max_mb


Média de latencia_p95
latencia_p95


Média de vazao_req_s
vazao_req_s


Média de cache_misses
cache_misses


Média de cache_hits
cache_hits

# SINCRONIZAÇÃO E FALHA CONTROLADA

**Mutexes** (threading.Lock()) foram implementados nas **funções da Cache** para evitar que múltiplas requisições sobrescrevam uma mesma chave simultaneamente.

A **fila** também possui **Mutex**, além de dois **semáforos** (para fila cheia e vazia) que colocam as threads para dormir, poupando a CPU.

Sem mecanismos de **exclusão mútua** (atualizacao_sem_bloqueio), o sistema registrou 30.000 atualizações **perdidas por condições de corrida**.

A implementação de **locks mitigou o erro** e garantiu zero perdas, e uma fila restrita a 2 lugares gerou 63 timeouts de inserção e 30 requisições rejeitadas.

Aplicar limites rigorosos de RAM (RLIMIT_AS) gerou exatamente 1 memoryerror, validando uma interrupção controlada no sistema.

# ANÁLISE DE MEMÓRIA E ARMAZENAMENTO

Uma leitura a **frio** sequencial em arquivos de 512MB demonstrou tempo de 0.227s com leitura convencional.

Na leitura via **mmap**, a mesma configuração levou 0.2445s, mas repassou o custo ao sistema com 1 falta de página grave e 593 menores.

Em **dados quentes** (em cache), o mmap superou a leitura convencional, diminuindo o tempo no indice_npy de 0.002573s para 0.0005843s.

A **latência de consulta** degradou significativamente (de 0.02176s para 0.2118s) quando o **corpus vetorial** expandiu de 315MB para 3157MB.

O **tempo de alocação** no cenário de **maior corpus** resultou em um salto para 120.600 faltas de página menores.

# RELAÇÃO COM IA GENERATIVA E LIMITAÇÕES

A **busca semântica em LLMs** depende totalmente da eficiência da memória virtual, do escalonamento de threads e da paginação gerida pelo Sistema Operacional.

Não utilizar ferramentas otimizadas, como o **mmap e semáforos**, causa sobrecarga de I/O e aumenta as **falhas de página** na aplicação.

Como limitação, o ambiente operou de forma **simulada e local**, não retratando latências reais de rede nem uso e concorrência gráfica em VRAM de clusters em produção.

A arquitetura se baseou em matrizes .npy e SQLite locais, fato que simplifica a complexidade e a serialização em comparação a bancos vetoriais distribuídos.